BANCO DE DADOS

T10
Exercícios práticos de normalização

1FN, 2FN, 3FN e BCNF com SQL e validação no MySQL

10 problemas hands-on | enunciados e resoluções agrupados

Prof. Calvetti

Competências exercitadas

Diagnosticar

Reconhecer grupos repetitivos e dependências inadequadas.

Normalizar

Aplicar 1FN, 2FN, 3FN e BCNF conforme as regras do domínio.

Implementar

Traduzir a decomposição para PKs, FKs, UNIQUEs e CHECKs.

Validar

Executar consultas que comprovem reconstrução e integridade.

Duas partes separadas

Parte I

Dez exercícios sem respostas, prontos para trabalho individual ou em grupo.

Parte II

Resoluções agrupadas com dependências, normalização, SQL e resultados.

Roteiro para cada exercício

- Leia a regra do domínio antes de observar os dados.
- Declare dependências funcionais e calcule as chaves candidatas.
- Aplique cada forma normal somente quando o diagnóstico justificar.
- Implemente o esquema no MySQL com constraints.
- Execute a consulta de validação e interprete o resultado.

01

Exercícios propostos

Os próximos vinte slides apresentam os problemas e as tarefas. As respostas aparecem somente na segunda parte.

Telefones do paciente em colunas repetidas

Uma clínica registra até três telefones na mesma linha do paciente.

id_paciente	nome	telefone1	telefone2	telefone3
1	Ana Lima	1191111	1192222	NULL
2	Bruno Reis	1193333	NULL	NULL
3	Carla Luz	1194444	1195555	1196666

Problema observado

A quantidade de telefones está embutida na estrutura. Novos números exigem nova coluna e muitas células permanecem vazias.

Diagnóstico e normalização

- Identifique a violação de 1FN e a chave da relação bruta.
- Proponha relações que permitam qualquer quantidade de telefones.
- Defina PKs e FKs no MySQL.
- Insira a massa de teste sem repetir o cadastro do paciente.
- Escreva uma consulta que reconstrua paciente e telefone.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Produtos armazenados como lista no pedido

Um sistema legado guarda códigos de produtos separados por vírgula.

id_pedido	data	id_cliente	produtos	quantidades
101	2026-10-01	10	7,9	2,1
102	2026-10-02	11	8	3
103	2026-10-03	10	7,8,9	1,2,1

Problema observado

Uma célula contém vários valores e depende da posição paralela em outra lista. Filtros e FKs ficam inviáveis.

Diagnóstico e normalização

- Explique por que produtos e quantidades violam 1FN.
- Separe cabeçalho e itens do pedido.
- Escolha a chave de ITEM_PEDIDO.
- Implemente FKs para cliente e produto.
- Consulte os itens do pedido 103.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Matrícula com dependências parciais

A secretaria mantém aluno, turma e nota na mesma relação.

id_aluno	nome_aluno	id_turma	semestre	nota
1	Ana Lima	501	2026.2	8,5
1	Ana Lima	502	2026.2	9,0
2	Bruno Reis	501	2026.2	7,0

Problema observado

A chave é composta, mas nome_aluno depende apenas de id_aluno e semestre depende apenas de id_turma.

Diagnóstico e normalização

- Determine a chave candidata e os atributos primos.
- Identifique dependências parciais.
- Decomponha até a 2FN.
- Mostre que as relações resultantes atendem 3FN.
- Reconstrua aluno, turma, semestre e nota.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Itens de compra com cliente e produto repetidos

Uma compra com vários produtos repete dados do cliente e do catálogo.

compra	cliente	nome_cliente	produto	nome_produto	qtd
900	10	Ana	7	Mouse	2
900	10	Ana	8	Teclado	1
901	11	Bruno	7	Mouse	3

Problema observado

A chave {compra, produto} determina a linha, mas vários atributos dependem apenas de compra, cliente ou produto.

Diagnóstico e normalização

- Declare as dependências funcionais.
- Separe dependências parciais na 2FN.
- Elimine dependências transitivas na 3FN.
- Implemente as relações com PKs e FKs.
- Calcule a quantidade total por compra.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Funcionário, departamento e gerente

O RH registra o departamento e o gerente dentro de cada funcionário.

id_func	nome	id_depto	depto	id_gerente	gerente
1	Lia	20	TI	7	Marcos
2	Caio	20	TI	7	Marcos
3	Rui	30	RH	8	Paula

Problema observado

O funcionário determina o departamento, que determina nome e gerente. O gerente também determina seu próprio nome.

Diagnóstico e normalização

- Identifique as dependências transitivas.
- Mostre por que a relação atende 2FN.
- Decomponha até a 3FN.
- Avalie BCNF nas relações obtidas.
- Liste funcionários com departamento e gerente.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Ordem de manutenção com cadastros misturados

Uma assistência técnica registra cliente, equipamento e técnico na ordem de serviço.

os	cliente	nome_cliente	equip	modelo	tecnico	nome_tecnico	horas
501	10	Ana	80	X100	7	Ivo	2,5
502	10	Ana	81	Z200	8	Bia	1,0
503	11	Rui	82	X100	7	Ivo	3,0

Problema observado

A OS tem chave simples, porém identificadores de cliente, equipamento e técnico determinam dados que se repetem.

Diagnóstico e normalização

- Declare as dependências funcionais.
- Avalie 1FN e 2FN.
- Elimine dependências transitivas.
- Defina CHECK para horas.
- Consulte horas trabalhadas por técnico.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Venda, produto e categoria

O relatório de vendas repete nomes de cliente, produto e categoria em cada item.

venda	cliente	nome_cliente	produto	nome_produto	categoria	nome_categoria	qtd
1001	10	Ana	7	Mouse	2	Periféricos	2
1001	10	Ana	8	Webcam	2	Periféricos	1
1002	11	Rui	9	Livro SQL	3	Livros	1

Problema observado

A chave é composta. Há dependências parciais e a categoria determina sua descrição.

Diagnóstico e normalização

- Determine a chave candidata.
- Aplique 2FN separando venda, produto e item.
- Aplique 3FN separando cliente e categoria.
- Defina restrições de quantidade.
- Reconstrua o relatório original.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Orientação acadêmica em 3FN fora da BCNF

Cada professor orienta uma única disciplina, e cada aluno pode ter orientações em várias disciplinas.

aluno	disciplina	professor
1	BD	Carla
1	IA	Diego
2	BD	Carla
3	Redes	Elisa

Problema observado

As dependências são {aluno, disciplina} -> professor e professor -> disciplina. Todos os atributos são primos.

Diagnóstico e normalização

- Calcule as duas chaves candidatas.
- Comprove que a relação atende 3FN.
- Identifique a violação de BCNF.
- Decomponha sem perdas.
- Reconstrua as orientações por JOIN.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Reserva de laboratório e responsável

Cada responsável atua em um único laboratório por turno. Uma turma pode reservar responsáveis diferentes.

turma	turno	responsavel	laboratorio
T1	NOITE	Ana	L201
T2	NOITE	Ana	L201
T3	MANHÃ	Bruno	L105
T1	MANHÃ	Carla	L110

Problema observado

As regras produzem duas candidatas e uma dependência cujo determinante não é superchave.

Diagnóstico e normalização

- Use as DFs para encontrar as chaves candidatas.
- Teste 3FN e BCNF.
- Escolha a DF que causa a decomposição.
- Demonstre a junção sem perdas.
- Consulte as reservas reconstruídas.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Histórico de pedidos e catálogo

Uma única relação reúne pedido, cliente, produto, fornecedor e categoria.

pedido	cliente	produto	categoria	fornecedor	qtd	preço
5001	Ana	Mouse	Perif.	Tech A	2	80,00
5001	Ana	Webcam	Perif.	Tech B	1	150,00
5002	Rui	Livro SQL	Livros	Editora X	1	90,00

Problema observado

A relação mistura cabeçalho, item e cadastros. Atualizações de categoria e fornecedor podem gerar contradições.

Diagnóstico e normalização

- Declare uma chave candidata para a linha do pedido.
- Aplique 1FN, 2FN e 3FN.
- Avalie BCNF em todas as relações.
- Implemente PKs, FKs, UNIQUEs e CHECKs.
- Reconstrua o histórico e calcule o total do pedido.

Entrega: memória de normalização, script SQL, massa de teste e consulta de validação com resultado.

Checklist antes de consultar o gabarito

- A chave candidata está justificada por fecho e minimalidade?
- Cada dependência funcional expressa uma regra válida do domínio?
- A decomposição removeu a violação sem descartar atributos?
- As PKs e FKs preservam a cardinalidade original?
- O JOIN recompõe os fatos sem combinações espúrias?
- A consulta de teste demonstra o comportamento esperado?

02

Resoluções práticas

Cada solução ocupa três slides: diagnóstico, formas normais e validação SQL com resultado.

Telefones do paciente em colunas repetidas

Dependência funcional	Leitura no domínio
id_paciente -> nome	O identificador define o cadastro do paciente.
id_paciente, telefone -> tipo	O tipo descreve cada telefone do paciente.

Chave candidata: id_paciente na tabela bruta. Na relação de telefones, {id_paciente, telefone}.

A violação decorre do grupo repetitivo telefone1, telefone2 e telefone3.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Colunas numeradas representam ocorrências.	Criar PACIENTE_TELEFONE com uma linha por número.
2FN	A chave de PACIENTE_TELEFONE é composta.	tipo depende da chave completa.
3FN	Nenhum não chave determina outro não chave.	As duas relações já atendem 3FN.
BCNF	Todo determinante é chave da própria relação.	As duas relações atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
PACIENTE	id_paciente PK, nome
PACIENTE_TELEFONE	id_paciente PK/FK, telefone PK, tipo

A normalização remove o limite artificial de três telefones e evita NULLs estruturais.

Validação prática da normalização

QUERY

```
SELECT p.id_paciente,p.nome,t.telefone,t.tipo
FROM paciente p
LEFT JOIN paciente_telefone t
 ON t.id_paciente=p.id_paciente
ORDER BY p.id_paciente,t.telefone;
```

Resultado da execução

id	paciente	telefone	tipo
1	Ana Lima	1191111	CELULAR
1	Ana Lima	1192222	RESIDENCIAL
2	Bruno Reis	1193333	CELULAR
3	Carla Luz	1194444	CELULAR

Cada telefone aparece como ocorrência independente. A clínica pode inserir novos números sem alterar a tabela.

Produtos armazenados como lista no pedido

Dependência funcional	Leitura no domínio
id_pedido -> data, id_cliente	O pedido define seu cabeçalho.
id_pedido, id_produto -> quantidade	A quantidade pertence à linha do pedido.
id_produto -> nome_produto	O produto define seu cadastro.

Chave candidata: id_pedido no cabeçalho e {id_pedido, id_produto} nos itens.

As listas produtos e quantidades não permitem valores elementares nem integridade referencial.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Listas em duas células.	Uma linha por produto em ITEM_PEDIDO.
2FN	Dados do pedido não ficam no item.	PEDIDO separado de ITEM_PEDIDO.
3FN	Nome do produto pertence ao produto.	PRODUTO recebe seus próprios dados.
BCNF	Determinantes são chaves locais.	PEDIDO, PRODUTO e ITEM_PEDIDO atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
PEDIDO	id_pedido PK, data, id_cliente FK
PRODUTO	id_produto PK, nome
ITEM_PEDIDO	id_pedido PK/FK, id_produto PK/FK, quantidade CHECK

A normalização permite validar produtos, somar quantidades e alterar a composição sem manipular strings.

Validação prática da normalização

QUERY

```
SELECT i.id_pedido,p.nome,i.quantidade
FROM item_pedido i
JOIN produto p ON p.id_produto=i.id_produto
WHERE i.id_pedido=103
ORDER BY p.id_produto;
```

Resultado da execução

pedido	produto	quantidade
103	Mouse	1
103	Teclado	2
103	Webcam	1

O resultado preserva a correspondência entre produto e quantidade sem depender da posição em uma lista.

Matrícula com dependências parciais

Dependência funcional	Leitura no domínio
id_aluno -> nome_aluno	O aluno define seu nome.
id_turma -> semestre	A turma define o semestre.
id_aluno, id_turma -> nota	A nota pertence à matrícula.

Chave candidata: {id_aluno, id_turma}. Ambos são atributos primos.

A relação atende 1FN, mas viola 2FN porque atributos não primos dependem de partes da candidata.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Valores simples e uma matrícula por linha.	A relação atende 1FN.
2FN	Há dependências parciais.	Criar ALUNO, TURMA e MATRICULA.
3FN	Não restam cadeias entre não chaves.	As três relações atendem 3FN.
BCNF	Cada determinante é chave local.	As três relações atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
ALUNO	id_aluno PK, nome
TURMA	id_turma PK, semestre
MATRICULA	id_aluno PK/FK, id_turma PK/FK, nota CHECK

A normalização altera o nome do aluno ou o semestre da turma em um único lugar.

Validação prática da normalização

QUERY

```
SELECT a.nome,t.id_turma,t.semestre,m.nota
FROM matricula m
JOIN aluno a ON a.id_aluno=m.id_aluno
JOIN turma t ON t.id_turma=m.id_turma
ORDER BY a.id_aluno,t.id_turma;
```

Resultado da execução

aluno	turma	semestre	nota
Ana Lima	501	2026.2	8,5
Ana Lima	502	2026.2	9,0
Bruno Reis	501	2026.2	7,0

O JOIN recompõe a visão original sem repetir o cadastro do aluno nem o semestre da turma.

Itens de compra com cliente e produto repetidos

Dependência funcional	Leitura no domínio
id_compra -> id_cliente	A compra pertence a um cliente.
id_cliente -> nome_cliente	O cliente define seu nome.
id_produto -> nome_produto	O produto define seu nome.
id_compra, id_produto -> quantidade	A quantidade pertence ao item.

Chave candidata: {id_compra, id_produto}.

A relação viola 2FN e 3FN. O cadastro do cliente também depende transitivamente da compra.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Todos os valores são simples.	A relação atende 1FN.
2FN	Produto e compra determinam fatos parciais.	Separar COMPRA, PRODUTO e ITEM_COMPRA.
3FN	compra determina cliente, que determina nome.	Criar CLIENTE.
BCNF	Determinantes tornam-se chaves locais.	As quatro relações atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
CLIENTE	id_cliente PK, nome
COMPRA	id_compra PK, id_cliente FK
PRODUTO	id_produto PK, nome
ITEM_COMPRA	id_compra PK/FK, id_produto PK/FK, quantidade

A normalização impede nomes conflitantes e preserva cada item por chave composta.

Validação prática da normalização

QUERY

```
SELECT c.id_compra,cl.nome,
 SUM(i.quantidade) AS itens
FROM compra c
JOIN cliente cl ON cl.id_cliente=c.id_cliente
JOIN item_compra i ON i.id_compra=c.id_compra
GROUP BY c.id_compra,cl.nome
ORDER BY c.id_compra;
```

Resultado da execução

compra	cliente	itens
900	Ana	3
901	Bruno	3

A agregação usa linhas de item válidas e o nome do cliente vem de uma única fonte.

Funcionário, departamento e gerente

Dependência funcional	Leitura no domínio
id_func -> nome, id_depto	O funcionário define seus dados.
id_depto -> depto, id_gerente	O departamento define nome e gerente.
id_gerente -> gerente	O gerente define seu nome.

Chave candidata: id_func.

A chave simples torna a 2FN automática, mas cadeias via departamento e gerente violam 3FN.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Valores simples.	A relação atende 1FN.
2FN	A chave candidata é simples.	A relação atende 2FN.
3FN	Há dependências transitivas.	Criar FUNCIONARIO, DEPARTAMENTO e GERENTE.
BCNF	Cada determinante é chave local.	As três relações atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
GERENTE	id_gerente PK, nome
DEPARTAMENTO	id_depto PK, nome, id_gerente FK
FUNCIONARIO	id_func PK, nome, id_depto FK

A normalização evita alterar o nome do gerente em todos os funcionários do departamento.

Validação prática da normalização

QUERY

```
SELECT f.nome AS funcionario,d.nome AS depto,
 g.nome AS gerente
FROM funcionario f
JOIN departamento d ON d.id_depto=f.id_depto
JOIN gerente g ON g.id_gerente=d.id_gerente
ORDER BY f.id_func;
```

Resultado da execução

funcionário	departamento	gerente
Lia	TI	Marcos
Caio	TI	Marcos
Rui	RH	Paula

A consulta reúne os fatos sem armazenar o gerente em cada funcionário.

Ordem de manutenção com cadastros misturados

Dependência funcional	Leitura no domínio
id_os -> id_cliente, id_equip, id_tecnico, horas	A OS identifica o atendimento.
id_cliente -> nome_cliente	O cliente define seu cadastro.
id_equip -> modelo, id_cliente	O equipamento define modelo e proprietário.
id_tecnico -> nome_tecnico	O técnico define seu nome.

Chave candidata: id_os.

A relação atende 2FN, mas viola 3FN por fatos de entidades diferentes dependentes da chave por cadeias.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Uma OS por linha e valores simples.	Atendida.
2FN	Chave simples.	Atendida automaticamente.
3FN	Cadastros dependem de identificadores não chave.	Criar CLIENTE, EQUIPAMENTO, TECNICO e OS.
BCNF	Determinantes são chaves locais.	Relações finais atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
CLIENTE	id_cliente PK, nome
EQUIPAMENTO	id_equip PK, modelo, id_cliente FK
TECNICO	id_tecnico PK, nome
ORDEM_SERVICO	id_os PK, id_equip FK, id_tecnico FK, horas CHECK

A normalização conserva o proprietário do equipamento e elimina nomes repetidos em ordens.

Validação prática da normalização

QUERY

```
SELECT t.nome,ROUND(SUM(o.horas),1) AS horas
FROM ordem_servico o
JOIN tecnico t ON t.id_tecnico=o.id_tecnico
GROUP BY t.id_tecnico,t.nome
ORDER BY t.id_tecnico;
```

Resultado da execução

técnico	horas
Ivo	5,5
Bia	1,0

As horas permanecem na ordem. O nome do técnico aparece apenas no cadastro.

Venda, produto e categoria

Dependência funcional	Leitura no domínio
id_venda -> id_cliente	A venda define o cliente.
id_cliente -> nome_cliente	O cliente define seu nome.
id_produto -> nome_produto, id_categoria	O produto define cadastro e categoria.
id_categoria -> nome_categoria	A categoria define seu nome.
id_venda, id_produto -> quantidade	O item define a quantidade.

Chave candidata: {id_venda, id_produto}.

A relação viola 2FN por dependências parciais e viola 3FN pelas cadeias cliente e categoria.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Valores simples.	Atendida.
2FN	Venda e produto determinam dados parciais.	Criar VENDA, PRODUTO e ITEM_VENDA.
3FN	Cliente e categoria determinam descrições.	Criar CLIENTE e CATEGORIA.
BCNF	Determinantes são chaves locais.	Cinco relações atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
CLIENTE	id_cliente PK, nome
CATEGORIA	id_categoria PK, nome UNIQUE
PRODUTO	id_produto PK, nome, id_categoria FK
VENDA	id_venda PK, id_cliente FK
ITEM_VENDA	id_venda PK/FK, id_produto PK/FK, quantidade CHECK

A normalização preserva a classificação do produto e evita grafias divergentes de categoria.

Validação prática da normalização

QUERY

```
SELECT v.id_venda,c.nome AS cliente,p.nome AS produto,
 cat.nome AS categoria,i.quantidade
FROM venda v JOIN cliente c ON c.id_cliente=v.id_cliente
JOIN item_venda i ON i.id_venda=v.id_venda
JOIN produto p ON p.id_produto=i.id_produto
JOIN categoria cat ON cat.id_categoria=p.id_categoria
ORDER BY v.id_venda,p.id_produto;
```

Resultado da execução

venda	cliente	produto	categoria	qtd
1001	Ana	Mouse	Periféricos	2
1001	Ana	Webcam	Periféricos	1
1002	Rui	Livro SQL	Livros	1

A visão final reúne cinco relações sem manter dados descritivos duplicados nos itens.

Orientação acadêmica em 3FN fora da BCNF

Dependência funcional	Leitura no domínio
aluno, disciplina -> professor	Um aluno tem um orientador por disciplina.
professor -> disciplina	Cada professor orienta uma única disciplina.

Chave candidata: {aluno, disciplina} e {aluno, professor}.

A DF professor -> disciplina atende a exceção da 3FN porque disciplina é primo, mas professor não é superchave.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Valores simples.	Atendida.
2FN	Não há atributo não primo.	Atendida.
3FN	O atributo determinado é primo.	Atendida pela condição formal.
BCNF	professor não é superchave.	Decompor em PROFESSOR_DISCIPLINA e ALUNO_PROFESSOR.

Esquema resultante

Relação	Chaves e atributos
PROFESSOR_DISCIPLINA	professor PK, disciplina
ALUNO_PROFESSOR	aluno PK, professor PK/FK

A BCNF impede associar o mesmo professor a disciplinas diferentes por linhas independentes.

Validação prática da normalização

QUERY

```
SELECT ap.aluno,pd.disciplina,ap.professor
FROM aluno_professor ap
JOIN professor_disciplina pd
 ON pd.professor=ap.professor
ORDER BY ap.aluno,pd.disciplina;
```

Resultado da execução

aluno	disciplina	professor
1	BD	Carla
1	IA	Diego
2	BD	Carla
3	Redes	Elisa

A interseção professor determina PROFESSOR_DISCIPLINA. A decomposição binária ocorre sem perdas.

Reserva de laboratório e responsável

Dependência funcional	Leitura no domínio
turma, turno -> responsavel	Uma turma tem um responsável por turno.
responsavel, turno -> laboratorio	O responsável atua em um laboratório no turno.

Chave candidata: {turma, turno} e {turma, responsavel, turno} não é mínima. Com as regras dadas, a única candidata é {turma, turno}.
A DF responsavel,turno -> laboratorio viola 3FN e BCNF porque o determinante não é superchave e laboratório não é primo.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Valores simples.	Atendida.
2FN	Não há dependência parcial para não primo.	Atendida.
3FN	responsavel,turno determina laboratório.	Decompor a dependência transitiva.
BCNF	O determinante vira chave local.	RESERVA e RESPONSAVEL_TURNO atendem BCNF.

Esquema resultante

Relação	Chaves e atributos
RESPONSAVEL_TURNO	responsavel PK, turno PK, laboratorio
RESERVA	turma PK, turno PK, responsavel FK composta

A normalização evita que o mesmo responsável apareça em dois laboratórios no mesmo turno.

Validação prática da normalização

QUERY

```
SELECT r.turma,r.turno,r.responsavel,rt.laboratorio
FROM reserva r
JOIN responsavel_turno rt
 ON rt.responsavel=r.responsavel AND rt.turno=r.turno
ORDER BY r.turma,r.turno;
```

Resultado da execução

turma	turno	responsável	laboratório
T1	MANHÃ	Carla	L110
T1	NOITE	Ana	L201
T2	NOITE	Ana	L201
T3	MANHÃ	Bruno	L105

A regra de laboratório por responsável e turno passa a ser garantida por uma chave composta.

Histórico de pedidos e catálogo

Dependência funcional	Leitura no domínio
id_pedido -> data, id_cliente	O pedido define seu cabeçalho.
id_cliente -> nome_cliente	O cliente define seu nome.
id_produto -> nome_produto, id_categoria, id_fornecedor	O produto define seu cadastro.
id_categoria -> nome_categoria	A categoria define sua descrição.
id_fornecedor -> nome_fornecedor	O fornecedor define seu nome.
id_pedido, id_produto -> quantidade, preco_praticado	A linha define quantidade e preço.

Chave candidata: {id_pedido, id_produto}.

Há dependências parciais e transitivas. A relação bruta produz anomalias de atualização, inserção e exclusão.

Decomposição e justificativa

Etapa	Diagnóstico	Resultado
1FN	Uma linha por item e valores simples.	Atendida na amostra.
2FN	Pedido e produto determinam fatos parciais.	Criar PEDIDO, PRODUTO e ITEM_PEDIDO.
3FN	Cliente, categoria e fornecedor determinam descrições.	Criar CLIENTE, CATEGORIA e FORNECEDOR.
BCNF	Cada determinante é chave local.	O esquema final atende BCNF.

Esquema resultante

Relação	Chaves e atributos
CLIENTE, CATEGORIA, FORNECEDOR	Cada cadastro com PK e nome UNIQUE quando aplicável
PRODUTO	id_produto PK, FKs de categoria e fornecedor
PEDIDO	id_pedido PK, id_cliente FK, data
ITEM_PEDIDO	id_pedido PK/FK, id_produto PK/FK, quantidade, preco_praticado

A normalização define uma fonte para cada fato e mantém o preço histórico no item do pedido.

Validação prática da normalização

QUERY

```
SELECT p.id_pedido,c.nome AS cliente,
 SUM(i.quantidade*i.preco_praticado) AS total
FROM pedido p JOIN cliente c ON c.id_cliente=p.id_cliente
JOIN item_pedido i ON i.id_pedido=p.id_pedido
GROUP BY p.id_pedido,c.nome
ORDER BY p.id_pedido;
```

Resultado da execução

pedido	cliente	total
5001	Ana	310,00
5002	Rui	90,00

O total resulta dos itens. O banco não precisa repetir nem armazenar o total sem uma justificativa de desempenho.

COMPARAÇÃO

O que cada forma normal controla

Forma	Pergunta de diagnóstico	Problema removido
1FN	Cada posição contém um valor do domínio?	Listas e grupos repetitivos
2FN	Um não primo depende de parte da candidata?	Dependências parciais
3FN	Um determinante não chave controla não primo?	Dependências transitivas
BCNF	Todo determinante não trivial é superchave?	Determinantes inadequados restantes

Consultas usadas nas soluções

Objetivo	Padrão SQL
Reconstrução	JOIN entre relações normalizadas
Duplicidade de candidata	GROUP BY da candidata e HAVING COUNT(*) > 1
Violação de dependência	COUNT(DISTINCT atributo determinado) por determinante
Integridade referencial	INSERT com FK inexistente deve falhar
Domínio	INSERT ou UPDATE fora do CHECK deve falhar

Por que normalizar

Atualização

Cada fato possui uma fonte definida e não diverge entre linhas.

Inserção

Entidades podem existir antes de participar de vínculos.

Exclusão

Apagar um vínculo não elimina cadastros independentes.

Integridade

Constraints locais implementam as regras declaradas.

RUBRICA

Critérios para correção

Critério	Evidência esperada
Dependências	Regras semânticas e não coincidências da amostra
Chaves	Fecho completo e minimalidade
Formas normais	Diagnóstico explícito em cada etapa
Esquema físico	PKs, FKs, UNIQUEs e CHECKs coerentes
Validação	Massa de teste, consulta e interpretação do resultado

Perguntas para discussão

- Por que uma chave simples torna a 2FN automática?
- Como uma relação pode atender 3FN e violar BCNF?
- Qual diferença existe entre junção sem perdas e preservação de dependências?
- Por que DISTINCT não corrige um modelo redundante?
- Quando uma redundância pode ser mantida de forma deliberada?

Produto da atividade

- Arquivo SQL com criação das relações e constraints.
- Massa de dados que reproduza a tabela bruta.
- Consultas de reconstrução e de detecção de violações.
- Memória curta com dependências, candidatas e justificativas.
- Registro dos erros provocados e da interpretação de cada mensagem do MySQL.

Normalização aplicada e verificável

Os dez casos mostram que normalizar exige compreender o domínio, declarar dependências e comprovar a reconstrução dos fatos.

A solução termina quando o modelo, o SQL e os testes expressam a mesma regra.